Reviewer Pack — Genus Inverse Proportionality to Disjointness Communication …

Entry 06752711f237 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-09 18:32:40 UTC

Genus Inverse Proportionality to Disjointness Communication Complexity

Entry ID: `06752711f237` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-09 18:32:40 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Geometry of Curves **Field B** (complexity object): Communication Complexity of DISJOINTNESS

Statement:

> For any n×n communication matrix M of the DISJOINTNESS problem, the genus g(M) of the curve defined by its entries satisfies g(M) ≤ 1/2 $\log_2$ CC(M), where CC(M) is the randomized communication complexity. Equality holds for all complete bipartite graphs.

Rationale (proposer's reasoning):

> The genus of a curve encodes topological constraints on its defining equations. By linking the genus to communication complexity, we expose geometric obstructions to efficient protocols, as higher genus implies more complex algebraic structure requiring greater communication.

Taxonomy category: `META_COMPLEXITY` (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `de02035b4e5009ff`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> default: >=80% seeds must support, no counterexample

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM₁	LLM₂
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
from fractions import Fraction
import math
import sys

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def gaussian_elimination(A, b):
        n = len(A)
        Augmented = [[A[i][j] for j in range(n)] + [b[i]] for i in range(n)]

        for i in range(n):
            max_row = i
            for k in range(i+1, n):
                if abs(Augmented[k][i]) > abs(Augmented[max_row][i]):
                    max_row = k

            Augmented[i], Augmented[max_row] = Augmented[max_row], Augmented[i]

            factor = Fraction(1, Augmented[i][i])
            for j in range(i, n+1):
                Augmented[i][j] *= factor

            for k in range(n):
                if k != i:
                    factor = Augmented[k][i]
                    for j in range(i, n+1):
                        Augmented[k][j] -= factor * Augmented[i][j]

        return [row[-1] for row in Augmented]

    def compute_genus(M):
        n = len(M)
        A = [[0]*n for _ in range(n)]
        b = [0]*n

        for i in range(n):
            for j in range(i+1, n):
```

```
            if M[i][j] == 1:
                A[i][j] = -1
                A[j][i] = -1
                b[i] += 1
                b[j] += 1

    try:
        x = gaussian_elimination(A, b)
    except ZeroDivisionError:
        return None

    genus = sum(x[i]**2 for i in range(n)) / 4
    return genus

def generate_disjointness_matrix(n):
    M = [[0]*n for _ in range(n)]
    for i in range(n):
        for j in range(i+1, n):
            M[i][j] = random.randint(0, 1)
            M[j][i] = M[i][j]
    return M

def communication_complexity(M):
    n = len(M)
    CC = 2 * sum(sum(row) for row in M) / (n * (n - 1))
    return CC

instances_tested = 0
total_metric_value = 0
conjecture_holds = True
counterexample = ""

n_values = [5, 10, 15, 20, 30, 40]
for n in n_values:
    for _ in range(5):  # Aim for at least 30 instances per seed
        M = generate_disjointness_matrix(n)
        CC = communication_complexity(M)

        if CC == 0:
            continue

        genus = compute_genus(M)
        if genus is None:
            counterexample = "mapping_undefined"
            conjecture_holds = False
            break

        metric_value = genus / math.log2(CC)
        total_metric_value += metric_value
        instances_tested += 1

mean_metric_value = total_metric_value / instances_tested if instances_tested > 0 else None
support_fraction = sum(1 for seed in range(30) if run_trial(seed)['conjecture_holds']) / 30

return {
    "metric_name": "g(M)/log₂ CC(M)",
```

```
        "metric_value": mean_metric_value,
        "instances_tested": instances_tested,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    seeds = [int(s) for s in sys.argv[1:]] or list(range(2, 30*2 + 1, 2))  # Default to first 30 p

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {{\"seed\": {seed}, \"metric_name\": \"{result['metric_name']}\", \"metric_
        results.append(result)

    mean_metric_value = sum(r['metric_value'] for r in results if r['metric_value'] is not None) /
    support_fraction = sum(1 for r in results if r['conjecture_holds']) / len(results)

    if all(r['conjecture_holds'] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std=0.0 support_fraction={support_fract
    elif any(not r['conjecture_holds'] for r in results) and any(r['counterexample'] != "mapping_u
        first_failing_seed = next(seed for seed, result in enumerate(results) if not result['conje
        print(f"RESULT: FALSIFIED counterexample=\"{next(result['counterexample'] for result in re
    else:
        print("RESULT: INCONCLUSIVE mapping_undefined")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_d2e19f50.py", line 115, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_d2e19f50.py", line 90, in run_trial
    genus = compute_genus(M)
            ^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_d2e19f50.py", line 65, in compute_genus
    b = gaussian_elimination(A, b)
        ^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_d2e19f50.py", line 46, in gaussian_elim
    factor = Augmented[j][i] / Augmented[i][i]
             ~~~~~~~~~~~~~~~~~^~~~~~~~~~~~~~~~
ZeroDivisionError: float division by zero
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: `INCONCLUSIVE`

Reasoning:

> Test crashed with division-by-zero error, preventing genus computation. Cannot verify conjecture's validity. | next: Debug gaussian_elimination implementation to handle singular matrices, then retest with validated inputs

11. Audit log (LLM calls)

Total LLM calls: 11

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	89920
2	propose	ollama_remote	qwen3:8b	0	0	112002
3	propose	ollama_remote	qwen3:8b	0	0	82208
4	propose	ollama_remote	qwen3:8b	0	0	61273
5	propose	ollama_remote	qwen3:8b	0	0	105844
6	preregistration	ollama_remote	qwen3:8b	0	0	24161
7	novelty	ollama_remote	qwen3:8b	0	0	20710
8	novelty	ollama_remote	qwen3:8b	0	0	17039
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17361
10	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13322
11	judge	ollama_remote	qwen3:8b	0	0	16470

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 560311 ms total latency. Provider mix: {'ollama_remote': 11}

(full prompt+response transcripts available in `research/audit/06752711f237.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/06752711f237.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/06752711f237.tar.gz` (if generated)